
ML - Machine Learning

Transformers

Riccardo Berta

2026.05.13

Contents

1	Transformers	1
1.1	Input Representation	2
1.1.1	Tokenisation	3
1.1.2	Embeddings	8
1.2	Attention	9
1.2.1	Queries, keys and values	9
1.2.2	Attention pooling	10
1.2.3	Interpretability	12
1.3	Scoring functions	15
1.3.1	Distance-based scoring	15
1.3.2	Learnable scoring	18
1.4	Machine Translation	20
1.4.1	Preprocessing	23
1.4.2	World-level Tokenization	24
1.4.3	Vocabulary	26
1.4.4	Pipeline	28
1.4.5	Word embeddings	30
1.4.6	Evaluation	32
1.5	An Encoder–Decoder solution	35
1.5.1	Encoder	36
1.5.2	Decoder	37
1.5.3	Seq2seq model	38
1.5.4	Loss function	39
1.5.5	Training loop	41
1.5.6	Prediction	46
1.5.7	Performance	48
1.6	Overcoming the bottleneck	48
1.6.1	Adding attention	49
1.6.2	Self-attention	59
1.7	The transformer architecture	59

1 Transformers

The success of deep learning has been driven largely by a small set of core architectures, namely **multilayer perceptrons**, **convolutional networks**, and **recurrent networks**. Despite the impres-

sive progress achieved with these models, their fundamental structure remained closely aligned with earlier formulations. Although a wide range of methodological improvements enriched the practitioner's toolkit (such as ReLU activations, residual connections, batch normalization, dropout, and adaptive optimization methods) the underlying architectures can still be understood as **scaled-up and refined versions of classical ideas**. In this perspective, the remarkable advances in deep learning are often attributed less to radically new model designs and more to external factors, in particular the **dramatic increase in computational capabilities**, enabled by parallel hardware such as GPUs, and the **availability of large-scale datasets**, which made it possible to effectively train increasingly expressive models.

However, a new architecture, the **Transformer**, has emerged as a dominant paradigm across a wide range of domains. Initially developed for natural language processing tasks such as machine translation, question answering, and text generation, it has since achieved state-of-the-art performance in areas as diverse as computer vision (e.g., image classification, object detection, segmentation) and spatio-temporal modeling (e.g., weather forecasting, traffic prediction, video understanding). The success of the Transformer stems from its **distinct architectural design**, which abandons both recurrence and convolution in favor of a **self-attention mechanism**. This mechanism enables the model to capture long-range dependencies and model complex, context-dependent interactions between input elements in a flexible and scalable way. Perhaps even more significantly, Transformers introduce a **unified modeling framework**. Once data is expressed in a suitable representation, the same architecture can be applied without substantial modification, regardless of whether the input consists of text, images, audio, or multimodal data. This shift reduces the reliance on specialized architectures tailored to specific modalities (such as convolutions for images or recurrent structures for sequences) and instead promotes a view in which all inputs are mapped into a **shared representation space**, where the model itself learns how to structure and exploit their relationships.

1.1 Input Representation

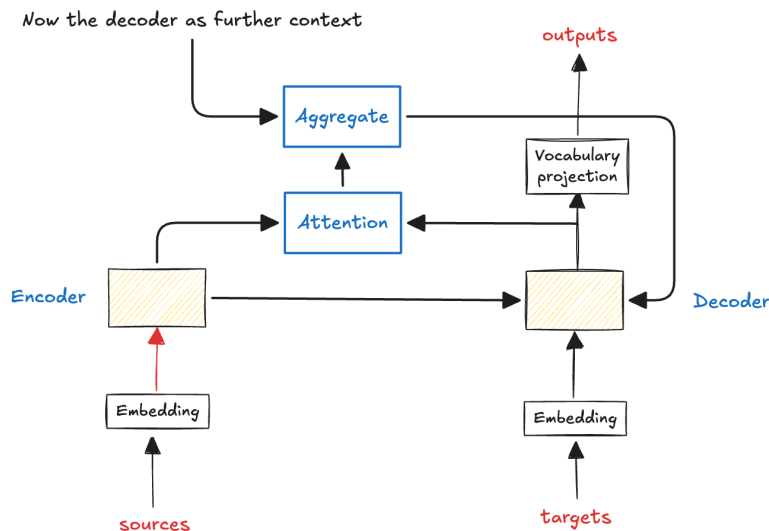
A Transformer does not operate directly on **raw inputs** such as characters, pixels, or audio samples. Instead, its native input consists of a **sequence of tokens**, each represented by a vector of size D :

$$x_i \in \mathbb{R}^D$$

By collecting N such tokens and stacking their vector representations row-wise, we obtain the input matrix:

$$X \in \mathbb{R}^{N \times D}$$

Each column corresponds to one **feature dimension** shared across all tokens



Although this matrix has the same shape as those used in many standard models, its interpretation is fundamentally different. In a typical MLP setting, each row corresponds to an independent data point and each column to an input feature with a fixed, predefined meaning. In a Transformer, instead, **each row represents a token within a single example**, and the columns no longer correspond to explicit features, but to the **coordinates of that token in a shared, learned representation space**. These dimensions do not have an individual semantic interpretation; rather, meaning is encoded collectively across the vector, and is learned from data rather than specified in advance.

The crucial point is that, from the Transformer's perspective, the input is simply the matrix X . Any modality-specific structure (such as word boundaries in text, spatial locality in images, or time-frequency patterns in audio) **must be handled before this stage**. This is achieved by first **tokenizing** the raw signal into discrete units (e.g., words, subwords, image patches, or audio frames), and then **embedding** each unit into a numerical vector. Once the matrix X is constructed, the Transformer operates on it without any explicit knowledge of the original data modality, effectively treating all inputs in a uniform representation space.

1.1.1 Tokenisation

Tokenisation is not a one-size-fits-all procedure: it depends on both the input modality and the task at hand. The underlying goal, however, remains the same: to transform a structured raw input into a collection of vectors of fixed dimension D , so that the Transformer can process them as a sequence of comparable units and apply the same operations to each of them. What varies across domains is therefore **what is chosen as a token** and **how it is mapped into a vector** representation.

In **text**, the most natural tokens are simply units of a sentence. A straightforward approach is to split a sentence into **words**, but in practice it is often more effective to use **subword** tokenization, where words are decomposed into smaller fragments. This allows the model to represent rare words, inflected forms, or proper names without requiring a separate vocabulary entry for every possible variant. After this segmentation step, each token is mapped to an integer index drawn from a **fixed vocabulary**. From this point onward, the model no longer operates on text, but on sequences of discrete identifiers that can be further transformed into vector representations. A simple example helps make this idea concrete. Suppose we start from the following **corpus**, that is, a collection of text data used to train or evaluate a model. A corpus typically consists of many sentences (or documents) drawn from some domain, and serves as the source from which the model learns patterns in language.

```
corpus = [  
    "the cat sat on the mat",  
    "the cat ate the rat",  
    "the rat sat on the mat",  
]
```

We begin by extracting the **vocabulary**, that is, the set of all distinct tokens that appear in the corpus. Each token is then assigned a unique integer identifier, allowing the text to be represented as a sequence of numerical indices that the model can process.

```
# collection of all words in the corpus  
words = []  
  
# extract words from each sentence and add them to the list  
for sentence in corpus:  
    words.extend(sentence.split())  
  
# remove duplicates and sort  
vocab_list = sorted(set(words))  
  
# assign an index to each word  
vocab = {}  
for idx, word in enumerate(vocab_list):  
    vocab[word] = idx  
  
print("Vocabulary:", vocab)
```

Vocabulary: {'ate': 0, 'cat': 1, 'mat': 2, 'on': 3, 'rat': 4, 'sat': 5, 'the': 6}

Given a new sentence, we can tokenize it by first splitting it into words and then replacing each word with its corresponding integer identifier from the vocabulary, thereby converting the sentence into a sequence of token IDs that the model can process.

```
# input sentence to be tokenized
sentence = "the cat sat on the mat"

# split the sentence into words
words = sentence.split()

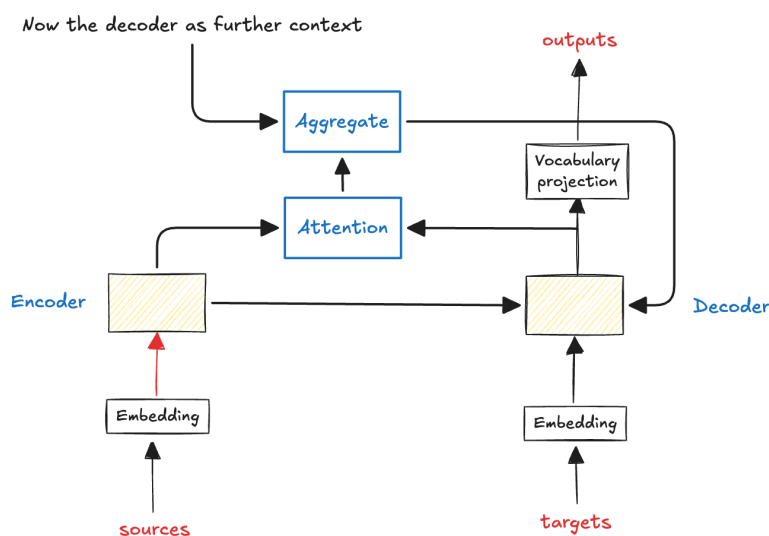
# convert each word to its corresponding id
token_ids = []

for w in words:
    token_ids.append(vocab[w])

print("Sentence:", sentence)
print("Token IDs:", token_ids)
print("N =", len(token_ids), "tokens")
```

Sentence: the cat sat on the mat
Token IDs: [6, 1, 5, 3, 6, 2]
N = 6 tokens

An **image** can be tokenized by dividing it into non-overlapping patches of size $P \times P$. Each patch contains a number of channels C (e.g., 3 for RGB images), and when the patch is flattened, it is transformed into a vector of length $C \times P \times P$.



As an example, the following function tokenizes a batch of images by splitting each image into non-overlapping patches. The input is assumed to have shape (batch size, number of channels, height, width). Each image is divided into square patches, so the resulting number of patches per image is:

$$N = \left\lfloor \frac{H}{P} \right\rfloor \times \left\lfloor \frac{W}{P} \right\rfloor$$

For each patch, all channel values are flattened into a single vector of length:

$$L = C \times P \times P$$

The output has shape (batch size, number of patches, patch size). This format is usually more convenient than keeping the patch dimension before the sequence dimension, because the model can treat the image as a sequence of N tokens, each with its own feature vector.

```
def image_to_patches(images, P):  
  
    # images: (batch, channels, height, width)  
    B, C, H, W = images.shape  
  
    # reshape into a grid of patches  
    patches = images.reshape(B, C, H//P, P, W//P, P)  
  
    # reorder dimensions so patches become a sequence  
    patches = patches.permute(0, 2, 4, 1, 3, 5)  
  
    # flatten each patch  
    patches = patches.reshape(B, -1, C*P*P)  
  
    return patches
```

In the following example, we construct a dummy batch consisting of two RGB images:

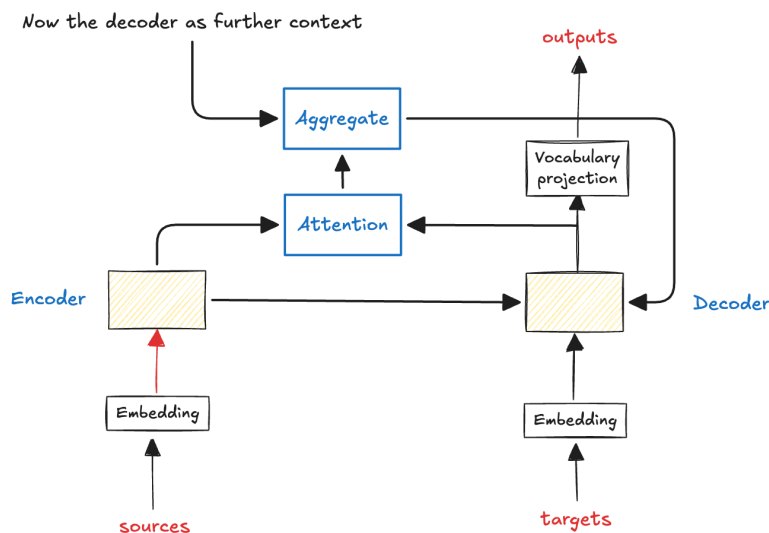
```
import torch  
  
# create a batch of 2 RGB images of size 224x224  
images = torch.randn(2, 3, 224, 224)  
  
# define the patch size  
P = 16  
  
# convert images into patches  
patches = image_to_patches(images, P)
```

```
print("Input shape:", images.shape)
print("Patches shape:", patches.shape)
```

```
Input shape: torch.Size([2, 3, 224, 224])
```

```
Patches shape: torch.Size([2, 196, 768])
```

Unlike text, which is naturally divided into words or subwords, or images, which can be partitioned into spatial patches, an **audio waveform** is a continuous temporal signal with **no intrinsic token boundaries**. Tokenizing audio therefore requires introducing an artificial segmentation in time. The signal is divided into short temporal regions that must be sufficiently compact to keep the sequence length computationally manageable, while still long enough to capture meaningful acoustic structures such as pitch, formants, harmonics, or transient events. The most common strategy is known as **framing**. The waveform is segmented into frames of fixed duration samples, typically corresponding to 20–40 ms of audio. Consecutive frames are generated by sliding a window along the signal with a fixed step size, called the **hop size**. When the hop size equals the frame length, the frames are adjacent and non-overlapping. More commonly, the hop size is smaller than the frame length, so consecutive frames partially overlap. This means that neighboring segments share part of the same waveform, producing a smoother and more redundant representation that preserves temporal continuity and reduces abrupt changes between frames.



Concretely, given an audio signal

$x[t]$

the i -th frame is defined as:

$$f_i = x[iH : iH + L] \quad i = 0, 1, \dots, N - 1$$

where N is the total number of frames, which depends on the total number of samples in the signal (T) and the choice of the frame length (L) and hop size (H):

$$N = \left\lfloor \frac{T - L}{H} \right\rfloor + 1$$

A widely used approach is to **tokenize a time–frequency representation** of the signal instead of the raw waveform. In practice, we first apply a Fourier transform to obtain a **spectrogram**, which represents the audio as a 2D array that can then be tokenized similarly to an image. The key intuition is that many perceptually relevant patterns (such as pitch, formants, harmonics, and transient onsets) become much more explicit in the spectrogram domain than in raw temporal samples. Consequently, the resulting tokens tend to encode more meaningful acoustic structure and higher-level semantic information.

1.1.2 Embeddings

Tokenization provides a collection of discrete symbolic units, but these units are not yet suitable for numerical computation. The role of embeddings is to **map each token into a vector** in a **shared D-dimensional space**, transforming heterogeneous inputs into a common numerical representation on which the same learning algorithms can operate. The key idea is that, once embedded, tokens originating from different modalities (or carrying different semantic meanings) become **comparable geometric objects**. Relationships such as similarity, context, or semantic structure can then be expressed through distances and directions in the embedding space. At a high level, an embedding can be viewed simply as a learned function:

$$e : \text{token} \rightarrow \mathbb{R}^D$$

Applying this mapping to the tokens produced during tokenization yields the input representation matrix:

$$X = \begin{bmatrix} e(t_1)^\top \\ e(t_2)^\top \\ \vdots \\ e(t_N)^\top \end{bmatrix} \in \mathbb{R}^{N \times D}$$

The key conceptual point is that embeddings do not merely encode tokens, they **define the geometry** in which it is possible to **measure compatibility**. When two tokens end up with similar embeddings, the model can easily make them attend to similar information, when embeddings are far apart, the model can keep them distinct. We will see several embedding strategies in the

examples, but the most common one is to simply learn a **lookup table** that maps each token to a vector. This is the standard approach for text, where we have a fixed vocabulary of tokens and we can learn an embedding matrix. For images and audio, we typically use a **learned linear projection** that maps raw pixel values or spectrogram values to the embedding space.

1.2 Attention

Up to this point, every architecture we have discussed relied (explicitly or implicitly) on the assumption that the **input lives in a fixed, well-defined shape**. In ImageNet classification, for example, we typically resize or crop images to 224x224 pixels, and the network is engineered (and trained) around that convention. In time-series forecasting, the situation is similar: an RNN is fed windows of a predetermined length, so “how much past” the model sees is fixed in advance. This design choice is convenient, but it becomes a **real bottleneck when the input is genuinely variable in size and in information content**. When sequences get long, the difficulty is not merely computational, it is **conceptual**: the model must decide what to retain, what to forget, and how to access something it saw (or produced) many steps ago. In practice, keeping track of “everything so far” becomes increasingly fragile as the context grows.

1.2.1 Queries, keys and values

What we need is a mechanism that can selectively read from a large collection of information, one where the amount of stored data can grow arbitrarily, yet the way we access it remains stable and efficient. A natural place to look for intuition is the **abstraction of a database**: a collection of **items**, each addressed by a **key** and carrying a **value**:

$$D = \{(k_1, v_1), (k_2, v_2), \dots, (k_m, v_m)\}$$

For instance, we can consider a database consisting of the last names and first names of a group of people, with the last name as the key and the first name as the value:

```
D = {"Zhang": "Aston",
     "Lipton": "Zachary",
     "Li": "Mu",
     "Smola": "Alex",
     "Hu": "Rachel",
     "Werness": "Brent"}
```

We can then **operate on the database** by issuing a **query**. For example, an exact query to retrieve a specific entry:

```
# Exact match
q = "Li"
print(D[q]) # Output: Mu
```

Mu

Or an approximate query to retrieve all entries whose key start with a given prefix:

```
# Approximate match (keys starting with q)
approx = {k: v for k, v in D.items() if k.startswith(q)}
print(f"Approximate matches: {approx}") # Output: {'Lipton': 'Zachary', 'Li': 'Mu'}
```

Approximate matches: {'Lipton': 'Zachary', 'Li': 'Mu'}

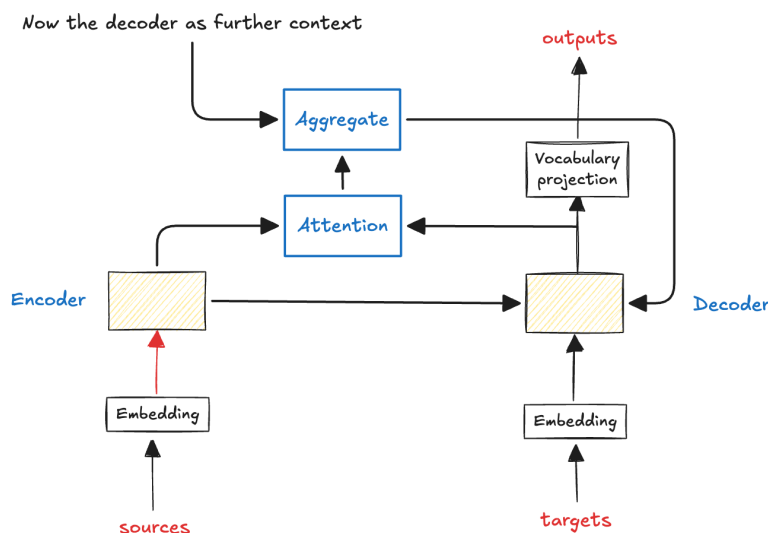
We can imagine more complex queries, such as retrieving the top-k entries most similar to a given text, or retrieving all entries that satisfy some logical condition. The crucial point is that **the size of the database can vary freely** (we can add or remove entries), however the **query mechanism is stable** (we do not need to change the way we query it).

1.2.2 Attention pooling

Even though the example is intentionally simple, it surfaces several important ideas. The first is that we can formulate a query as an operation on key–value pairs in a way that does **not depend on the database size**: the same mathematical object still makes sense whether the database contains a handful of items or millions. The second is that the very **same query can legitimately produce different outputs**, because the answer is not “hard-coded” in the procedure but is induced by the current contents of the database. The third is that operating over an enormous state space **does not require complicated code**: the mechanism can be as elementary as exact lookup, approximate matching, or retrieving the top-k most relevant entries, with the **complexity pushed into the representation and the similarity score** rather than into bespoke control logic. Finally, and perhaps most importantly, none of this requires **compressing or simplifying the database** just to make retrieval workable: we can keep a large collection of stored information and still read from it effectively by learning how to address it. This perspective leads directly to one of the most influential architectural ideas of the last decade: the **attention mechanism**, which can be defined as:

$$A(q, D) = \sum_{i=1}^m a(q, k_i) v_i$$

Here the terms $a()$ are **scalar attention weights** that quantify how strongly the query matches each key. The term attention is natural: the mechanism assigns large weights only to a small subset of entries, and the output is therefore dominated by the corresponding values, those are the items the model is effectively "paying attention" to. Operationally, attention reads from the database by producing a weighted linear combination of the stored values:



The operation of computing a weighted sum of values guided by how well the query matches each key is called **attention pooling**. Like other pooling operations, it aggregates information, but here the aggregation is **adaptive**, shaped by the query rather than fixed in advance. The earlier exact-lookup example is recovered as a limiting case: if one key matches perfectly, its weight becomes 1 while all others go to 0, and the sum collapses to the single retrieved value. Several useful special cases follow from different choices of weights and similarity scores.

A natural next step is to **constrain** the weights so that they truly represent an allocation of attention over the database entries. Formally, we impose the **nonnegativity constraint**:

$$a(q, k_i) \geq 0 \quad \text{for all } i$$

thereby ensuring that attention weights can only enhance or suppress contributions through selection, rather than through cancellation. Without this constraint, the retrieval mechanism could exhibit pathological behavior by effectively "anti-attending" to certain entries, subtracting their associated values from the final result. From the perspective of memory retrieval, such negative coefficients are difficult to interpret. When accessing a memory, we may choose to ignore irrelevant entries, but we do not retrieve the negation of their contents. Nonnegative attention weights therefore enforce a more coherent interpretation of attention as a distribution of relevance over stored information.

We additionally impose the **sum-to-one constraint**:

$$\sum_{i=1}^m a(q, k_i) = 1$$

so that the attention weights acquire the interpretation of a probability distribution over the database entries. Under this constraint, the mechanism must decide **how to distribute a fixed budget of attention** across all keys. This normalization is important both conceptually and computationally. Conceptually, it reinforces the interpretation of attention as selective allocation of relevance. Computationally, it prevents the mechanism from arbitrarily increasing the magnitude of the output simply by scaling all weights simultaneously; the overall gain remains controlled because increasing attention on one entry necessarily reduces it elsewhere. A standard way to enforce both nonnegativity and normalization is through the softmax function. Starting from an unnormalized compatibility score s , which measures the similarity between the query and the i -th key, attention weights are defined as

$$a(q, k_i) = \frac{\exp(s(q, k_i))}{\sum_{j=1}^m \exp(s(q, k_j))}$$

The exponential guarantees positivity, while the normalization term ensures that all weights sum to one. As a result, entries with higher compatibility scores receive a larger fraction of the available attention mass.

1.2.3 Interpretability

One of the reasons attention mechanisms are so compelling is that they often appear **naturally interpretable**. Because the attention weights are constrained to be nonnegative and to sum to one, they can be viewed as a distribution describing **how strongly the model attends to each key**. In this interpretation, large weights indicate entries that are highly relevant to the current query, whereas small weights correspond to entries that contribute little to the final retrieval. This perspective is particularly appealing because it frames attention as **a soft, content-based retrieval process**: rather than applying an opaque transformation, the model appears to “look up” the information that matters most for the current computation. At the same time, it is important to maintain the correct interpretation of these weights. Attention scores are not explanations explicitly generated by the network, they are simply intermediate computational quantities that happen to admit an intuitive semantic reading. In some cases they correlate strongly with what humans would regard as relevance, but this alignment is neither guaranteed nor necessarily stable across architectures or tasks. Nevertheless, the explicit structure of attention provides **a valuable analytical window into the model’s behavior**. By visualizing how the attention distribution changes as the query varies, we can often gain substantial insight into the retrieval patterns, dependencies, and internal organization learned by the network. To make these attention patterns easier to inspect, we define a `show_heatmap` function that visualizes the attention

weights as a **heatmap**, placing queries along the vertical axis and keys along the horizontal axis. In this representation, each cell encodes how strongly a given query attends to a particular key, allowing us to immediately **identify concentration patterns, long-range dependencies, and regions of focused or diffuse attention**.

```
import matplotlib.pyplot as plt

def show_heatmap(matrix, xlabel, ylabel, title=None):

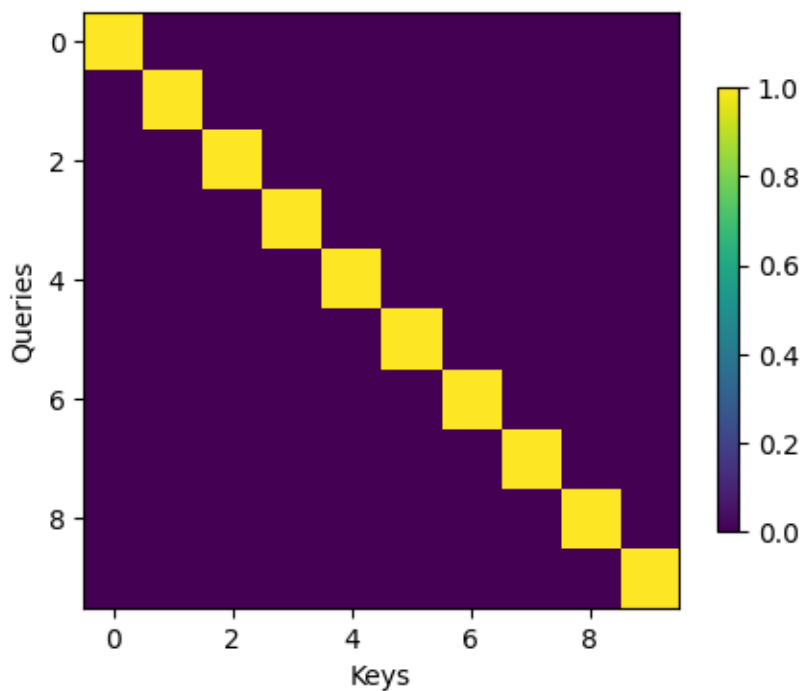
    fig, ax = plt.subplots(figsize=(5, 5))

    pcm = ax.imshow(matrix.detach().numpy())
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    if title:
        ax.set_title(title)

    fig.colorbar(pcm, ax=ax, shrink=0.6)
    plt.show()
```

As a simple sanity check, let us visualize the **identity matrix**, which corresponds to the idealized case in which **each query assigns all its attention to the matching key and zero attention to every other entry**. In this setting, the attention weight is equal to 1 only when the query and key coincide.

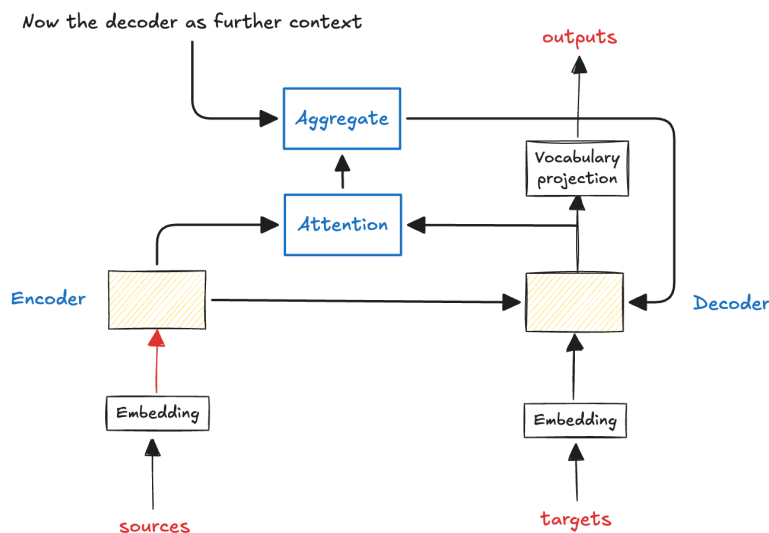
```
attention_weights = torch.eye(10)
show_heatmap(attention_weights, xlabel='Keys', ylabel='Queries')
```



What makes the attention mechanism particularly remarkable is the contrast between the **simplicity of its computation** and **the scale of the space over which it operates**. The underlying procedure (the query together with its similarity function) can remain extremely compact even when the number of keys and values becomes enormous. From the perspective of neural network design, this is a highly desirable property. The mechanism does not require a separate set of learned parameters for every possible memory entry, instead, it reuses the same learned matching rule to dynamically access a potentially vast database. As the number of entries grows, the model must indeed compute more similarity scores and more attention weights, so the runtime cost in computation and memory increases accordingly. However, this increase is computational rather than parametric: the model does not acquire additional degrees of freedom simply because the database becomes larger. The attention weights are generated on the fly by a fixed compatibility function, rather than being stored as independently learned parameters. Equally important, the mechanism itself does not need to be redesigned as the database scales. Attention applies exactly the same retrieval and aggregation procedure whether the memory contains tens, thousands, or millions of entries. **Increasing the database size merely introduces more candidates against which the query must be compared**, it does not alter the structure of the computation. In this sense, attention provides a **scalable and size-agnostic framework** for content-based information retrieval and aggregation.

1.3 Scoring functions

An attention layer is essentially determined once we specify **how to measure the compatibility between a query and a key**. Everything else follows a standard and reusable computational pipeline. For each key, the mechanism first computes a compatibility score that quantifies how relevant that entry is to the current query. These raw scores are then normalized through a softmax operation, producing attention weights that are nonnegative and sum to one. The resulting weights can therefore be interpreted as a distribution of relevance over the available entries. Finally, the layer retrieves information from the database by forming a weighted combination of the corresponding values, so that entries receiving higher attention contribute more strongly to the output representation:



This is precisely the purpose of the **scoring function**:

$$s(q, k)$$

which takes a query and a key and returns a single scalar measuring their compatibility.

1.3.1 Distance-based scoring

A conceptually simple way to build such a score is to start from **distance**:

$$d(q, k) = \|q - k\|^2$$

and convert it into a similarity score by negating (and often scaling) it:

$$s(q, k) = -\|q - k\|^2$$

This directly encodes the intuitive principle that "closer means more relevant": keys located near the query in representation space receive larger scores and therefore larger attention weights. Although this formulation is easy to interpret geometrically, it is **not the most computationally efficient choice**. Computing distances and norms is generally less hardware-friendly than the primitive operation modern accelerators are optimized for: matrix multiplication. For this reason, most attention mechanisms in deep learning rely on scoring functions based on dot products, which can be evaluated extremely efficiently and integrate naturally into highly optimized linear-algebra pipelines.

The distance-based similarity score can be rewritten in a form that makes its connection to dot products explicit:

$$s(q, k) = -\|q - k\|^2 = -\|q\|^2 - \|k\|^2 + 2q^\top k$$

The first term depends only on the query and is therefore identical for every key. Adding or subtracting the same constant from all scores does not change the resulting distribution. Consequently, this term can be removed without affecting the attention mechanism:

$$s(q, k) = -\|k\|^2 + 2q^\top k$$

In many deep architectures, activations are intentionally kept within a controlled range through techniques such as batch normalization or layer normalization. Under these conditions, the norms of the keys often vary relatively little, making this term like a small offset which is commonly discarded:

$$s(q, k) = 2q^\top k$$

The multiplicative factor of 2 is itself irrelevant, since uniformly rescaling all scores only changes the temperature of the softmax. This leads to the standard dot-product formulation:

$$s(q, k) = q^\top k$$

One final issue concerns numerical stability. Suppose:

$$q \in \mathbb{R}^d, \quad k \in \mathbb{R}^d$$

If the components of q and k are modeled as independent random variables with zero mean and unit variance, then the dot product has variance proportional to d . As the dimensionality increases, the scores therefore become increasingly large in magnitude. Feeding such values directly into a softmax tends to produce extremely sharp distributions in which one key dominates almost completely, while the remaining weights collapse toward zero. To keep the scale of the scores roughly independent of dimension, we use **scaled dot-product score** dividing by square root of the dimensionality:

$$s(q, k) = \frac{q^\top k}{\sqrt{d}}$$

We can implement a batch version of this function as follows using PyTorch:

```
import torch.nn as nn
import torch.nn.functional as F

class ScaledDotProductAttention(nn.Module):

    def __init__(self):
        super().__init__()

    # queries : (batch, num_queries, d)
    # keys    : (batch, num_keys, d)
    # values  : (batch, num_keys, d_v)
    def forward(self, queries, keys, values):

        # Find the dimension of the queries/keys
        d = queries.shape[-1]

        # Compute the attention scores
        scores = queries @ keys.transpose(-2, -1) / (d ** 0.5)

        # Normalize the scores to obtain attention weights
        self.weights = F.softmax(scores, dim=-1)

        # return the weighted sum of the values
        return self.weights @ values
```

We have assumed that queries and keys belong to the same vector space, meaning that they share the same dimensionality. While convenient, this assumption is not fundamental to the attention mechanism itself. In many practical settings, queries and keys naturally arise from different representations and may therefore have incompatible dimensions. The obstacle is that the dot product is no longer directly defined when the vectors have different dimensions. A standard solution is to learn two linear projections that map queries and keys into a **common latent space** of dimension h . Compatibility is then computed in this shared representation space rather than in the original coordinate systems. Formally, we introduce two learnable matrices:

$$W_q \in \mathbb{R}^{h \times d_q} \quad W_k \in \mathbb{R}^{h \times d_k}$$

and define the projected representations:

$$\tilde{q} = W_q q \quad \tilde{k} = W_k k$$

The attention score is then computed as:

$$s(q, k) = \frac{\tilde{q}^\top \tilde{k}}{\sqrt{h}} = \frac{(W_q q)^\top (W_k k)}{\sqrt{h}}$$

These learned projections allow the model not only to reconcile dimensional mismatches, but also to discover a representation space in which compatibility relationships become easier to express and optimize.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class GeneralizedScaledDotProductAttention(nn.Module):

    def __init__(self, d_q, d_k, h):
        super().__init__()
        self.W_q = nn.Linear(d_q, h, bias=False) # projects queries into R^h
        self.W_k = nn.Linear(d_k, h, bias=False) # projects keys into R^h
        self.h = h

    # queries : (batch, num_queries, d_q)
    # keys : (batch, num_keys, d_k)
    # values : (batch, num_keys, d_v) ← d_v is unrestricted
    def forward(self, queries, keys, values):

        q = self.W_q(queries) # (batch, num_queries, h)
        k = self.W_k(keys) # (batch, num_keys, h)

        scores = q @ k.transpose(-2, -1) / (self.h ** 0.5)
        self.weights = self.dropout(F.softmax(scores, dim=-1))

        return self.weights @ values # (batch, num_queries, d_v)
```

1.3.2 Learnable scoring

The scaled dot-product score is efficient and well-motivated, but **it encodes a specific assumption** about what “compatibility” means: two vectors are similar when they point in the same direction. This works well in many cases, but it is a structural constraint: the score is entirely determined by the geometric alignment of the projected query and key vectors. However, the relevant relationship is not necessarily one of geometric alignment, but of **learned semantic compatibility**. In these situations, we would like the scoring function itself to be a **learnable component**, one that can discover from data what “relevant” means for the task at hand. This

is the idea behind the **additive score**. Rather than combining the projected query and key by multiplication, we first sum them and then pass the result through a nonlinearity, letting a small learned function shape their interaction:

$$s(q, k) = w^\top \tanh(W_q q + W_k k + b)$$

where

$$b \in \mathbb{R}^h \quad w \in \mathbb{R}^h$$

This gives the mechanism greater expressive flexibility, since the notion of “match” is no longer restricted to correlations captured by an inner product.

```
import torch.nn as nn

class AdditiveAttention(nn.Module):
    def __init__(self, d_q, d_k, h):
        super().__init__()
        self.W_q = nn.Linear(d_q, h, bias=True)
        self.W_k = nn.Linear(d_k, h, bias=False)
        self.w_o = nn.Linear(h, 1, bias=False)

        # queries : (batch, num_queries, d_q)
        # keys    : (batch, num_keys, d_k)
        # values  : (batch, num_keys, d_v)

    def forward(self, queries, keys, values):

        q = self.W_q(queries).unsqueeze(2) # (batch, num_queries, 1, h)
        k = self.W_k(keys).unsqueeze(1)   # (batch, 1, num_keys, h)
        scores = self.w_o(torch.tanh(q + k)).squeeze(-1) # (batch, num_queries,
        ↪ num_keys)
        self.weights = F.softmax(scores, dim=-1)
        return self.weights @ values        # (batch, num_queries, d_v)
```

Attention scoring functions provide a flexible mechanism for aggregating information over sequences of variable length. Among them, scaled dot-product attention has become the fundamental building block of modern Transformer architectures due to both its simplicity and its computational efficiency on parallel hardware. When queries and keys originate from representations with different dimensionalities, this framework remains fully applicable. Both generalized dot-product attention and additive attention address the mismatch by learning projections into a shared latent space before computing compatibility scores. In this way, attention retains the same conceptual structure while accommodating heterogeneous representa-

tions. Because attention layers sit at the computational core of contemporary large-scale models, improving their efficiency and stability has become a major area of research and engineering. Recent advances focus on optimized kernels, memory-efficient implementations, mixed-precision computation, and hardware-aware execution strategies. A representative example is the **NVIDIA Transformer Engine library**, which provides highly optimized implementations of Transformer components for modern GPU architectures.

1.4 Machine Translation

The abstract framework developed so far (queries, keys, values, and scoring functions) becomes particularly compelling when applied to problems that genuinely require a **flexible, size-independent mechanism for retrieving and aggregating information**. Machine translation is a canonical example of such a problem. In machine translation, the objective is to **transform a sentence in a source language into a semantically equivalent sentence in a target language**. Both the input and the output are variable-length sequences, and, crucially, there is generally no fixed positional correspondence between their elements. A word appearing early in the source sentence may influence a word generated much later in the target sentence, and different languages may reorder grammatical structures in radically different ways. More broadly, machine translation exemplifies a large class of problems involving **mappings between two unaligned sequences**. The challenge is not simply to process sequential data, but to dynamically determine which parts of one sequence are most relevant to each element of the other. Attention mechanisms are particularly well suited to this setting because they allow the model to perform content-based retrieval over the entire input sequence while generating each output element. As a concrete instance of this task, we work with the **English–Italian dataset** from the **Tatoeba Project**, a crowd-sourced collection of bilingual sentence pairs.

```
import urllib.request
import zipfile
import os

# Define the URL of the dataset
url = "https://www.dropbox.com/scl/fi/csvxhe8q50h7fwa67p7bh/eng-
    ita.zip?rlkey=y7m93phovbbdadg00l02yau658st=fl7bpvfv&dl=1"

# Create data directory if it does not exist
os.makedirs("./data", exist_ok=True)

# Prepare data if not already existing
if not os.path.exists("./data/eng-ita.zip"):
```

```
print("Downloading dataset ... ")

# Download the dataset
u = urllib.request.urlopen(url)
data = u.read()
u.close()

# Save the dataset to a local file
with open("./data/eng-ita.zip", "wb") as f :
    f.write(data)

# Extract the dataset
with zipfile.ZipFile("./data/eng-ita.zip","r") as zip_ref:
    zip_ref.extractall("./data")

else:
    print("Dataset already exists. Skipping download.")
```

Dataset already exists. Skipping download.

Each example in the dataset consists of a tab-separated pair of an English source sequence and its Italian translation, accompanied by a CC-BY attribution field that we discard.

```
import pandas as pd

# Load the English-Italian sentence pairs into a Pandas DataFrame
eng_ita = pd.read_csv("./data/ita.txt", sep="\t", header=None,
                      names=["english", "italian", "attribution"],
                      encoding="utf-8")

# Drop the attribution column (CC-BY licence metadata, not needed for training)
eng_ita = eng_ita.drop("attribution", axis=1)

# Drop duplicate pairs
eng_ita = eng_ita.drop_duplicates()

# Reset index after deduplication
eng_ita = eng_ita.reset_index(drop=True)

# Show some statistics about the dataset
eng_ita.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 403477 entries, 0 to 403476
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   english     403477 non-null  object
1   italian     403477 non-null  object
dtypes: object(2)
memory usage: 6.2+ MB
```

It can be useful to inspect a few sentence pairs from the dataset in order to develop an intuition for the structure and variability of the translation task. Examining concrete examples helps highlight important characteristics of the data, such as differences in sentence length, word order, and grammatical structure between the source and target languages.

```
samples = eng_ita.sample(5)

for i, row in enumerate(samples.itertuples(index=False), 1):
    print(f"Example {i}")
    print(f"English : {row.english}")
    print(f"Italian : {row.italian}")
    print("-" * 60)
```

Example 1

English : But I can't understand them very well.

Italian : Ma non riesco a capirli molto bene.

Example 2

English : That's what Tom and Mary did.

Italian : È quello che hanno fatto Tom e Mary.

Example 3

English : Don't forget that Tom is blind.

Italian : Non scordate che Tom è cieco.

Example 4

English : I've had a tough year.

Italian : Ho avuto un anno difficile.

Example 5

English : This is my son, Tom.

Italian : Questo è mio figlio, Tom.

We fix the seed of the random number generator in order to reproduce the results:

```
import torch;
import numpy as np;
import random;

# set the seeds for reproducibility of results
seed = 42;

torch.manual_seed(seed);
np.random.seed(seed);
random.seed(seed);
```

1.4.1 Preprocessing

After downloading the dataset, we apply a series of preprocessing steps to normalize the raw text. These transformations are designed to reduce unnecessary variability and to make tokenization more consistent. For example, non-breaking spaces are replaced with standard spaces, uppercase characters are converted to lowercase, and punctuation marks are separated from adjacent words by inserting whitespace. This ensures that punctuation is treated as an independent token rather than being merged with surrounding text.

```
import re

def preprocess(text):
    text = text.replace('\u00a0', ' ')          # non-breaking space
    text = text.lower()                          # lowercase
    text = re.sub(r'([^\w\s])', r' \1 ', text)  # space around punctuation
    text = re.sub(r'\s+', ' ', text).strip()    # normalize spaces
    return text
```

We can apply this preprocessing to both columns of the dataset:


```
eng_ita["english"] = eng_ita["english"].apply(preprocess)
eng_ita["italian"] = eng_ita["italian"].apply(preprocess)
```

1.4.2 World-level Tokenization

To keep the exposition simple, we adopt a **word-level tokenization strategy**, although modern neural translation systems typically employ more sophisticated approaches, such as subword tokenization. Under this scheme, each token corresponds either to a word or to an individual punctuation symbol. The preprocessing pipeline operates on pairs of aligned sentences. Each sentence is first decomposed into a sequence of tokens, after which a special **end-of-sequence token** (`<eos>`) is appended explicitly to the end of the sequence. The procedure ultimately produces two collections of tokenized sequences, each element in these collections is itself a list of tokens. More specifically, `src[i]` contains the tokenized form of the *i*-th source sentence (English in this example), while `tgt[i]` contains the tokenized representation of the corresponding aligned target sentence (Italian):

```
def tokenize(data, max_examples=None):
    src = [row.split() + ['<eos>'] for row in data["english"][:max_examples]]
    tgt = [row.split() + ['<eos>'] for row in data["italian"][:max_examples]]
    return src, tgt
```

As a concrete example, we apply this preprocessing pipeline to the sentence pairs in the dataset and inspect a randomly selected tokenized example to visualize the resulting sequence representation.

```
eng, ita = tokenize(eng_ita)

random_idx = random.randrange(len(eng))

print(eng[random_idx])
print(ita[random_idx])
```

```
['tom', 'works', 'at', 'the', 'local', 'supermarket', '.', '<eos>']
['tom', 'lavora', 'nel', 'supermercato', 'locale', '.', '<eos>']
```

To better understand the structure of the dataset, let us visualize the distribution of sequence lengths by plotting a histogram of the number of tokens per sentence. In this relatively simple English–Italian translation corpus, the vast majority of sequences contain fewer than 20 tokens.

```
import matplotlib.pyplot as plt

# Compute sentence lengths in terms of number of tokens
eng_lengths = [len(tokens) for tokens in eng]
ita_lengths = [len(tokens) for tokens in ita]

# Prepare bins and counts for the histogram
bins = range(1, 20)
eng_counts = [eng_lengths.count(i) for i in bins]
ita_counts = [ita_lengths.count(i) for i in bins]

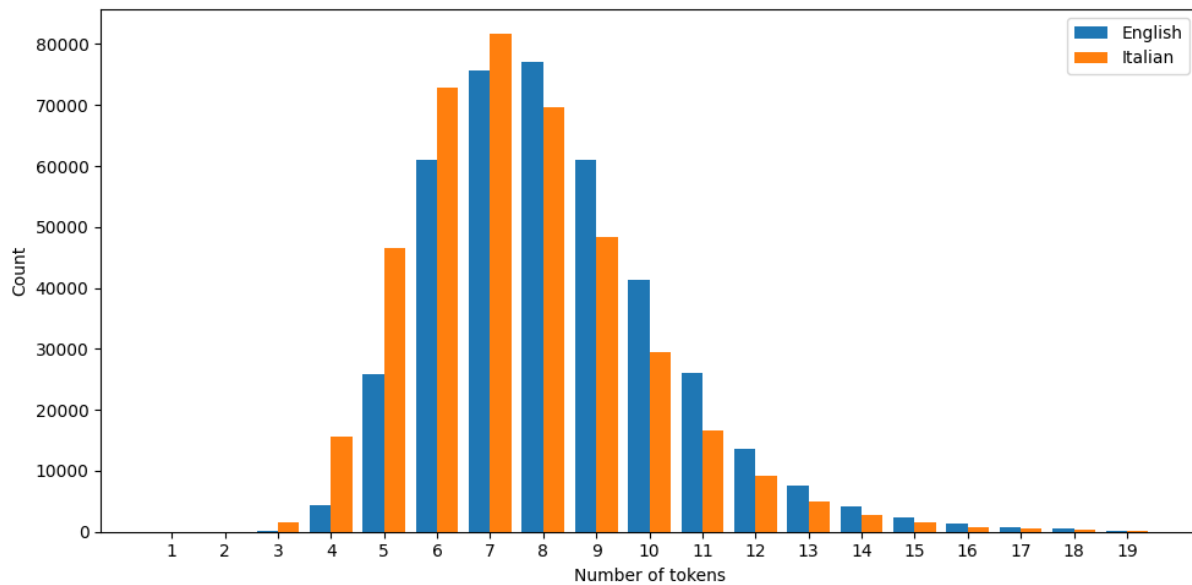
# Define the x-axis positions for the bars
x = np.arange(len(bins))

# Set the width of the bars
width = 0.4

# Plot the histogram
fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(x - width/2, eng_counts, width, label="English")
ax.bar(x + width/2, ita_counts, width, label="Italian")

# Customize the x-axis ticks and labels
ax.set_xticks(x)
ax.set_xticklabels(bins)
ax.set_xlabel("Number of tokens")
ax.set_ylabel("Count")
ax.legend()

plt.tight_layout()
plt.show()
```



1.4.3 Vocabulary

Since the dataset consists of aligned sentence pairs in two different languages, we construct **two distinct vocabularies**: one for the source language and one for the target language. Under a word-level tokenization scheme, the number of unique tokens can grow rapidly because each surface word form is treated as a separate vocabulary entry. To keep the vocabulary size manageable and reduce sparsity, tokens appearing fewer than two times in the corpus are replaced by a shared **unknown token** (). In addition to ordinary words, the vocabularies include several special-purpose tokens used to structure and control the sequence-processing pipeline. One important example is the **beginning-of-sequence token** (), which marks the start of a target sentence. During decoding, this token acts as the initial input to the model, signaling that generation of a new sequence should begin. Other special tokens include the **end-of-sequence token** (), already considered and the **padding token** (), which is used to standardize sequence lengths within a minibatch. If a sequence is shorter than the desired length, tokens are appended to its end until the required size is reached. Conversely, sequences that exceed the maximum allowed length are truncated by discarding tokens beyond the cutoff point. This procedure ensures that all sequences within the same minibatch share a common shape and can therefore be processed efficiently in parallel. Finally, we also record the true length of each source sequence excluding padding tokens. This information is often important during training and evaluation, since padding should not contribute to the model's representation or loss computation.

```
from collections import Counter

def build_vocab(tokens,
                min_freq=2,
                special_tokens=('<unk>', '<pad>', '<bos>', '<eos>')):

    # Count token frequencies
    counter = Counter(
        token
        for sequence in tokens
        for token in sequence
        if token not in special_tokens
    )

    # Build vocabulary
    vocab = list(special_tokens) + [
        token for token, freq in counter.items()
        if freq >= min_freq
    ]

    # Create mappings from tokens to indices and vice versa
    token_to_idx = {token: idx for idx, token in enumerate(vocab)}
    idx_to_token = {idx: token for token, idx in token_to_idx.items()}

    return token_to_idx, idx_to_token
```

The function returns two complementary dictionaries: one maps tokens to integer indices, allowing text sequences to be converted into numerical representations suitable for the model, while the other performs the inverse mapping, transforming predicted indices back into readable text. Since the dataset contains two different languages, we construct one vocabulary pair for the source language and another for the target language:

```
eng_to_idx, idx_to_eng = build_vocab(eng)
ita_to_idx, idx_to_ita = build_vocab(ita)

print(f"English vocabulary size: {len(eng_to_idx)}")
print(f"Italian vocabulary size: {len(ita_to_idx)}")
```

English vocabulary size: 11726

Italian vocabulary size: 21442

These dictionaries allow us to convert a text sequence into a sequence of token IDs suitable for numerical processing, and conversely to reconstruct readable text from a sequence of predicted token IDs.

```
def encode(sequence, vocab, max_len):
    unk, pad = vocab['<unk>'], vocab['<pad>']
    ids = [vocab.get(t, unk) for t in sequence][:max_len]
    ids += [pad] * (max_len - len(ids))
    return ids

def decode(indices, vocab):
    return [vocab[i] for i in indices if vocab[i] != '<pad>']
```

For example, we can take a tokenized source sentence, convert it into a sequence of token IDs using the encode function, and then apply the inverse mapping to recover the original text:

```
tokens = ['the', 'cat', '<eos>']
print(tokens)

indices = encode(tokens, eng_to_idx, max_len=10)
print(indices)

decoded = decode(indices, idx_to_eng)
print(decoded)
```

```
['the', 'cat', '<eos>']
[845, 330, 3, 1, 1, 1, 1, 1, 1, 1]
['the', 'cat', '<eos>']
```

1.4.4 Pipeline

At this point we have all the individual ingredients in place: a preprocessed corpus, a tokenizer, and two vocabularies that map words to integer indices. We can assemble these pieces into a coherent data pipeline, one that can efficiently produce batches of training examples ready to be consumed by a model. We will exploit the Dataset class (to define how individual examples are accessed and transformed) and DataLoader classes (to handle batching, shuffling, and parallel data loading) from PyTorch:

```

from torch.utils.data import Dataset, DataLoader, random_split

class TranslationDataset(Dataset):

    # Provide source and target sequences and vocabularies to the constructor
    def __init__(self, src_sequences, tgt_sequences, src_vocab, tgt_vocab,
                  max_src_len=20, max_tgt_len=20):
        self.tgt_bos = tgt_vocab['<bos>']
        self.src = self._encode(src_sequences, src_vocab, max_src_len)
        self.tgt = self._encode(tgt_sequences, tgt_vocab, max_tgt_len)

    # Internal method to encode a list of token sequences into a tensor of token IDs
    def _encode(self, sequences, vocab, max_len):
        return torch.tensor([encode(seq, vocab, max_len) for seq in sequences],
                             dtype=torch.long)

    # Return the number of examples in the dataset
    def __len__(self):
        return len(self.src)

    # Return a specific example from the dataset
    def __getitem__(self, idx):
        src = self.src[idx] # (max_src_len,)
        tgt = self.tgt[idx] # (max_tgt_len,)
        dec_in = torch.cat([torch.tensor([self.tgt_bos]), tgt[:-1]]) # <bos> +
        ↪ tgt[:-1]
        dec_out = tgt # target
        return src, dec_in, dec_out

```

We instantiate the dataset from the full corpus and split it into a training set and a validation set:

```

eng_ita_dataset = TranslationDataset(eng, ita, eng_to_idx, ita_to_idx)
eng_ita_train_size = int(0.9 * len(eng_ita_dataset))
eng_ita_val_size = len(eng_ita_dataset) - eng_ita_train_size
eng_ita_train_set, eng_ita_val_set = random_split(eng_ita_dataset,
        ↪ [eng_ita_train_size, eng_ita_val_size])

print(f"Training samples: {eng_ita_train_size:,}")
print(f"Validation samples: {eng_ita_val_size:,}")

```

Training samples: 363,129

Validation samples: 40,348

Finally, we wrap each split in a DataLoader:

```
eng_ita_train_loader = DataLoader(eng_ita_train_set, batch_size=128, shuffle=True)
eng_ita_val_loader   = DataLoader(eng_ita_val_set,   batch_size=128, shuffle=False)
```

With the pipeline in place, sentences are now represented as a sequence of vectors, a numeric format that a machine learning model can operate on.

1.4.5 Word embeddings

However, token indices are merely arbitrary integer identifiers and carry no intrinsic geometric or semantic structure. In the space of integers, token 42 is not meaningfully “closer” to token 41 than to token 9000. Since neural networks operate through algebraic operations (matrix multiplications, weighted sums, distances, dot products) and these operations implicitly assume that the numerical inputs possess some meaningful geometric structure. Raw token IDs do not satisfy this assumption. An **embedding layer** addresses this problem by associating each token with a **learnable dense vector representation**. Concretely, the layer maintains a parameter matrix:

$$E \in \mathbb{R}^{V \times D}$$

where V is the vocabulary size and D is the embedding dimension. Each row i is the vector representation of the token with index i . Looking up the embedding for a token is simply selecting the corresponding row:

$$e(\text{token}_i) = E[i] \in \mathbb{R}^D$$

The embedding vectors are typically initialized randomly and optimized jointly with the rest of the network during training. Over time, the model learns a geometry in which tokens appearing in similar linguistic contexts acquire similar vector representations. This is a simple implementation:

```
class Embedding(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()

        # Initialize the embedding matrix as a learnable parameter
        self.weight = nn.Parameter(torch.randn(vocab_size, embed_dim))
```

```
# idx : (batch, seq_len)
def forward(self, idx):

    # Look up the embedding vectors for the given token indices
    return self.weight[idx] # (batch, seq_len, embed_dim)
```

As a simple example, we instantiate an embedding layer and use it to retrieve the vector representations associated with two tokens from the vocabulary. Initially, these vectors are random, since the embedding matrix has not yet been trained, but during optimization the model will gradually organize them into a meaningful geometric representation space.

```
# Size of the vocabulary
V = len(eng_to_idx)

# Select
D = 4

# Create an embedding layer
E = Embedding(vocab_size=V, embed_dim=D)
print("Embedding matrix shape:", E.weight.shape)

# Look up embeddings for two tokens using the existing vocabulary
cat_id = torch.tensor([eng_to_idx["cat"]])
dog_id = torch.tensor([eng_to_idx["dog"]])

cat_embedded = E(cat_id)
dog_embedded = E(dog_id)

print("Embedding of 'cat':", cat_embedded)
print("Embedding of 'dog':", dog_embedded)
```

```
Embedding matrix shape: torch.Size([11726, 4])
Embedding of 'cat': tensor([[ 0.4303,  0.3305,  1.3761, -0.7184]], grad_fn=<IndexBackward0>)
Embedding of 'dog': tensor([[ -1.8478,  0.4687,  0.9946, -0.6954]], grad_fn=<IndexBackward0>)
```

For example, after training, we would generally expect the embedding vectors associated with semantically related words such as “cat” and “dog” to lie closer to one another in the embedding space than to the vector associated with an unrelated concept such as “run”.

1.4.6 Evaluation

Defining a problem also means **deciding how to measure success**. For machine translation, this is less straightforward than it might seem: a sentence can have many valid translations, and comparing two strings character by character would penalise perfectly correct outputs that simply chose different words. The standard metric is **Bilingual Evaluation Understudy (BLEU)**, which measures how much of the predicted output appears in the reference translation, at multiple levels of granularity. The key building block is the notion of an **n-gram**, a contiguous subsequence of exactly n tokens. For example, given the sentence:

"the cat is on the mat"

Its 1-grams (unigrams), 2-grams (bigrams), and 3-grams (trigrams) are:

- 1-grams: "the", "cat", "is", "on", "the", "mat"
- 2-grams: "the cat", "cat is", "is on", "on the", "the mat"
- 3-grams: "the cat is", "cat is on", "is on the", "on the mat"

Longer n -grams **capture more context** and impose **stricter ordering constraints**: matching a 4-gram requires getting four consecutive words right, not just individual tokens. The core idea of BLEU is to measure, for each order n , how many n -grams in the predicted sequence also appear in the reference:

$$p_n = \frac{\sum_{\mathbf{s} \in G_n(\hat{\mathbf{y}})} \min(c(\mathbf{s}, \hat{\mathbf{y}}), c(\mathbf{s}, \mathbf{y}))}{\sum_{\mathbf{s} \in G_n(\hat{\mathbf{y}})} c(\mathbf{s}, \hat{\mathbf{y}})}$$

where G_n is the set of distinct n -grams in a sentence and $c()$ counts how many times an n -gram occurs. The minimum operation in the numerator is a **clipping mechanism**: without it, a model could inflate the score by repeating a common word many times ("the the the the"). Clipping ensures that a predicted n -gram is credited at most as many times as it appears in the reference. Here an implementation in Python:

```
from collections import Counter

def ngram_precision(prediction, target, n):

    # Count n-grams in the prediction
    pred_ngrams = Counter(tuple(prediction[i:i+n]) for i in range(len(prediction) - n
↵ + 1))

    # Count n-grams in the target
    target_ngrams = Counter(tuple(target[i:i+n]) for i in range(len(target) -
↵ n + 1))
```

```

# Compute the count of n-grams in the prediction that also appear in the target
matched = sum(min(c, target_ngrams[s]) for s, c in pred_ngrams.items())

# Compute the total number of n-grams in the prediction
total = sum(pred_ngrams.values())

# Compute the precision
precision = matched / total if total > 0 else 0.0

# Return the precision, matched and total
return precision

```

Taking the earlier example, where the reference is "the cat is on the mat" and the prediction is "the cat is on the roof", the precisions at each order are:

n	Matched n-grams	p_n
1	"the", "cat", "is", "on", "the" → 5 of 6	5/6
2	"the cat", "cat is", "is on", "on the" → 4 of 5	4/5
3	"the cat is", "cat is on", "is on the" → 3 of 4	3/4
4	"the cat is on", "cat is on the" → 2 of 3	2/3

The scores drop as n grows because the single mismatch at the end ("roof" instead of "mat") contaminates progressively longer windows. Precision alone has a **blind spot**: it rewards every correct word but never asks what was missed. A prediction of just "the" scores a perfect unigram precision against almost any reference, despite conveying nothing. The natural fix would be recall, but recall is awkward when multiple valid translations of different lengths exist. BLEU instead applies a **brevity penalty** that discounts scores when the prediction is suspiciously short:

$$BP = \begin{cases} 1 & \text{if } |\hat{y}| \geq |y| \\ \exp\left(1 - \frac{|y|}{|\hat{y}|}\right) & \text{if } |\hat{y}| < |y| \end{cases}$$

There is no penalty when the prediction is at least as long as the reference. Below that threshold the penalty decays smoothly toward zero: the shorter the prediction, the harsher the discount.

```
import math

def brevity_penalty(predictions, targets):

    # Compute the total length of the predicted and target sequences
    pred_len = sum(len(pred) for pred in predictions)
    target_len = sum(len(tgt) for tgt in targets)

    # If the predicted length is greater than or equal to the target length,
    # return a penalty of 1.0 (no penalty)
    if pred_len ≥ target_len:
        return 1.0

    # Otherwise, return a penalty that decays exponentially as the predicted
    # length falls short of the target length
    return math.exp(1 - target_len / pred_len)
```

Combining the brevity penalty with precisions across all orders gives the final BLEU score:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N \frac{1}{N} \log p_n\right)$$

The precisions are combined as a geometric mean rather than an arithmetic mean. Unlike the arithmetic mean, which averages values additively, the geometric mean multiplies them, so a single near-zero precision drags the whole product toward zero, no matter how well the other orders perform. This ensures that a catastrophically low score at any single order drags the overall score down significantly, so the model cannot ignore any level of matching.

```
def bleu(predictions, targets, N=4):

    # Compute the brevity penalty for the predictions and targets
    bp = brevity_penalty(predictions, targets)

    # Compute the precision for each n-gram order and store them in a list
    precisions = []
    for n in range(1, N + 1):
        pn = sum(ngram_precision(pred, tgt, n) for pred, tgt in zip(predictions,
↪ targets)) / len(predictions)
        precisions.append(pn)

    # Compute the logarithm of the precisions
    p_log_sum = sum((1 / N) * math.log(pn) if pn > 0 else float('-inf') for pn in
↪ precisions)
```

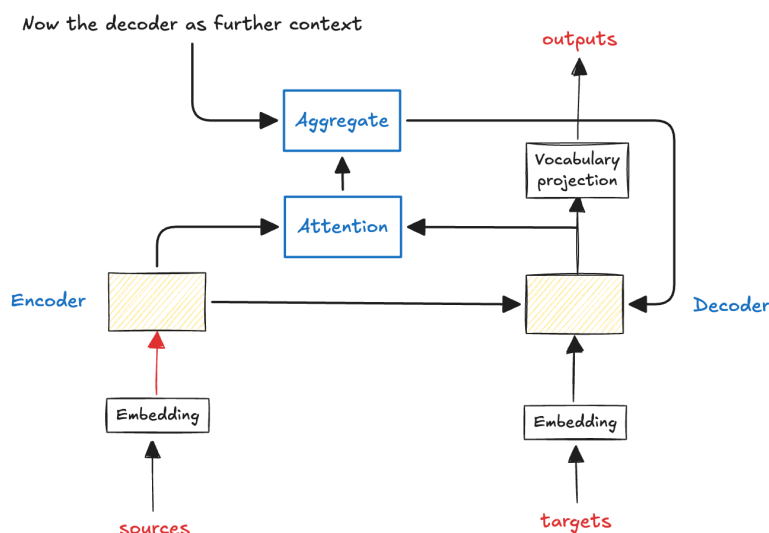
```
# Combine the brevity penalty with the geometric mean of the precisions to get the  
↳ final BLEU score  
return bp * math.exp(p_log_sum)
```

A BLEU score of 1 indicates a perfect match, a score of 0 means no overlap at any order. Short n-grams capture **vocabulary accuracy**, longer ones capture whether the model preserves **correct phrasing**, so a good BLEU score requires getting both individual words and their local ordering right.

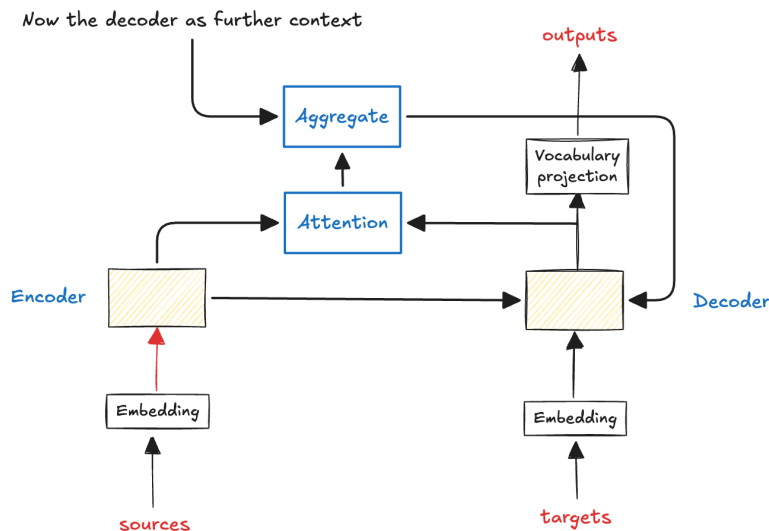
With the problem fully specified (data, preprocessing, and evaluation metric) the remaining question is **what model to use**. We start with a recurrent encoder–decoder, and then introduce attention to overcome its limitations.

1.5 An Encoder–Decoder solution

The **encoder–decoder architecture**, introduced for sequence-to-sequence learning, consists of two networks working in tandem. The **encoder** reads the input sequence and compresses it into an internal representation. The **decoder** then generates the output sequence one token at a time, using that representation together with the tokens it has already produced, as illustrated below:



For instance, given the English input []"He", "is", "watching", "."], the encoder compresses it into a single representation, and the decoder uses it to produce the Italian translation token by token ["Egli", "guarda", "."] as shown below:



Now, let's consider how to implement this architecture concretely.

1.5.1 Encoder

The encoder reads the input sequence:

$$x_1, x_2, \dots, x_T$$

and at each time step updates its hidden state:

$$h_t = f(x_t, h_{t-1})$$

The **context variable** is then derived from the full collection of hidden states:

$$c = q(h_1, h_2, \dots, h_T)$$

The simplest choice, and the one used here, is to take the final hidden state as the context:

$$c = h_T$$

This **compresses** the entire input sequence into a **single fixed-size vector**. More expressive alternatives exist, for instance, bidirectional RNNs process the sequence in both directions, giving each position access to both past and future context. However, for clarity we proceed with the simplest formulation. In the following implementation, the encoder stacks our custom embedding layer and a multi-layer GRU:

```
class Encoder(nn.Module):

    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers, dropout=0.0):
        super().__init__()
```

```
# Create an embedding layer to convert token IDs into dense vectors
self.embedding = Embedding(vocab_size, embed_dim)

# Create a multi-layer GRU to process the embedded input sequence and produce
↳ hidden states
self.rnn = nn.GRU(embed_dim, hidden_dim, num_layers,
                  batch_first=True,
                  dropout=dropout if num_layers > 1 else 0)

def forward(self, X):
    emb = self.embedding(X)
    output, state = self.rnn(emb)
    return output, state
```

1.5.2 Decoder

The decoder takes the encoder's final hidden state as its initial hidden state and generates the target sequence one token at a time, embedding each input token before passing it to the GRU and projecting the output onto the target vocabulary:

```
import torch.nn as nn

class Decoder(nn.Module):

    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers, dropout=0.0):
        super().__init__()

        # Create an embedding layer to convert token IDs into dense vectors
        self.embedding = Embedding(vocab_size, embed_dim)

        # Create a multi-layer GRU to process the embedded input sequence and produce
        ↳ hidden states
        self.rnn = nn.GRU(embed_dim, hidden_dim, num_layers,
                          batch_first=True,
                          dropout=dropout if num_layers > 1 else 0)

        # Create a linear layer to project the hidden states to the vocabulary size
        # for output prediction
        self.fc = nn.Linear(hidden_dim, vocab_size)
```

```
def forward(self, Y, state):
    emb = self.embedding(Y)
    output, state = self.rnn(emb, state)
    logits = self.fc(output)
    return logits, state
```

1.5.3 Seq2seq model

Now we can implement the entire architecture as a single PyTorch module:

```
class Seq2Seq(nn.Module):

    def __init__(self, encoder, decoder):
        super().__init__()

        # Store the encoder
        self.encoder = encoder

        # Store the decoder
        self.decoder = decoder

    def forward(self, X, Y):
        _, state = self.encoder(X)
        logits, _ = self.decoder(Y, state)
        return logits
```

During inference, the decoder operates **autoregressively**: starting from the "" token, it generates one token at a time, feeding each newly produced token back into the model in order to predict the next one (see the previous image). This iterative process continues until the decoder emits the "" token, signaling that the sequence is complete. Training, however, is performed differently. Instead of relying on previously generated predictions, we can provide the decoder with the correct target token at each step using a strategy known as **teacher forcing**. Because the entire target sequence is available in advance, all decoding steps can be processed simultaneously, enabling efficient parallel computation across the sequence dimension. Let's instantiate a model to be used for the translation task:

```
# Instantiate the encoder
encoder = Encoder(vocab_size=len(eng_to_idx), embed_dim=256, hidden_dim=512,
    ↪ num_layers=2, dropout=0.3)
```

```
# Instantiate the decoder
decoder = Decoder(vocab_size=len(ita_to_idx), embed_dim=256, hidden_dim=512,
                 num_layers=2, dropout=0.3)

# Instantiate the seq2seq model
seq2seq_model = Seq2Seq(encoder, decoder)
```

1.5.4 Loss function

At each decoding step, the model outputs a vector of logits:

$$z \in \mathbb{R}^V$$

over the target vocabulary, from which a probability distribution is obtained via softmax:

$$p_j = \frac{\exp(z_j)}{\sum_{k=1}^V \exp(z_k)}$$

Predicting the correct next token out of several candidates is a **multi-class classification problem**, making **cross-entropy loss** the natural training objective. At each position, it penalises the model by how little probability mass it assigned to the correct token:

$$\ell = -\log p_y = -z_y + \log \sum_{k=1}^V \exp(z_k)$$

A practical complication arises because sentences in the same minibatch generally have different lengths. To enable efficient batch processing, shorter sequences are **padded** with the special "<pad>" token so that all sequences share a common length. For example, if a batch contains the target sentences:

["Amo", "i", "gatti"]

["Il", "cane", "corre", "velocemente"]

the shorter sentence is padded as:

["Amo", "i", "gatti", "<pad>"]

Padding positions carry no meaningful prediction target, so including them in the loss would dilute the gradient signal. They must be excluded, so that the final loss averages only over the positions containing real tokens:

$$\mathcal{L} = -\frac{1}{|\{(b, t) : y_{b,t} \neq \text{pad}\}|} \sum_b \sum_{t: y_{b,t} \neq \text{pad}} \log p_{y_{b,t}}$$

We can implement this loss function in Python as follows:


```
def cross_entropy_loss(logits, target):  
  
    # Create a mask to ignore padding tokens in the loss computation  
    mask = target != ita_to_idx['<pad>']  
  
    # Compute the log probabilities of the predicted logits  
    log_probs = F.log_softmax(logits, dim=-1)  
  
    # Gather the log probabilities corresponding to the correct target tokens  
    correct = log_probs.gather(-1, target.unsqueeze(-1)).squeeze(-1)  
  
    #  
    return -correct[mask].mean()
```

We jointly optimize all parameters of both the encoder and decoder using the Adam optimizer, a widely adopted default choice that automatically adapts the effective learning rate for each parameter during training. We use a standard initial learning rate:

```
optimizer = torch.optim.Adam(seq2seq_model.parameters(), lr=1e-3)
```

Before starting training, it is often useful to compute the **total number of trainable parameters**, all weights and biases updated by the optimizer during learning. This provides a rough indication of the **model's representational capacity** as well as its **computational and memory requirements**:

```
total_params = sum(p.numel() for p in seq2seq_model.parameters() if p.requires_grad)  
print(f"Model parameters: {total_params:,}")
```

Model parameters: 25,008,066

With roughly 25 million trainable parameters, the model is **moderately sized**. A substantial fraction of these parameters comes from the two embedding layers (one for each language) together with the weights of the stacked GRU layers. This scale is large enough to capture meaningful translation patterns and nontrivial linguistic structure in the dataset, while still remaining compact enough to train in a reasonable amount of time on a modest GPU.

1.5.5 Training loop

Before starting training, we check whether an accelerated computation device is available so that the model can be moved to a GPU or, on Apple Silicon systems, to an MPS backend for faster execution:

```
# Check for available device (GPU, MPS, or CPU)
device = torch.device(
    'cuda' if torch.cuda.is_available() else
    'mps'  if torch.backends.mps.is_available() else
    'cpu'
)

# Send the model to the selected device
seq2seq_model = seq2seq_model.to(device)

# Show the selected device
print(f"Using device: {device}")
```

Using device: mps

The training loop iterates over minibatches, performs a forward pass, computes the loss, clips gradients to norm 1.0 to prevent the instability common in recurrent networks, and updates the parameters. It returns the average loss over the epoch:

```
def train(model, loader):

    # Set the model to training mode
    model.train()

    # Initialize a variable to accumulate the total loss across all batches
    total_loss = 0

    # Iterate over the batches of data provided by the loader
    for src, dec_in, dec_out in loader:

        # Move the source, decoder input, and decoder output tensors to the selected
        ↪ device
        src = src.to(device)
        dec_in = dec_in.to(device)
        dec_out = dec_out.to(device)
```

```
# Clear the gradients of all model parameters to prepare for the backward pass
optimizer.zero_grad()

# Perform a forward pass through the model to obtain the predicted logits for
↳ the current batch
logits = model(src, dec_in)

# Compute the loss by comparing the predicted logits with the actual decoder
↳ output
loss = cross_entropy_loss(logits, dec_out)

# Perform a backward pass to compute the gradients of the loss with respect
# to all model parameters
loss.backward()

# Clip the gradients to a maximum norm of 1.0 to prevent exploding gradients
↳ during training
nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

# Update the model parameters using the optimizer based on the computed
↳ gradients
optimizer.step()

# Accumulate the loss for the current batch into the total loss variable
total_loss += loss.item()

# After processing all batches, compute the average loss by dividing the total
↳ loss by
# the number of batches
return total_loss / len(loader)
```

The evaluation function mirrors the training loop but with two key differences: `model.eval()` switches off dropout and other training-specific behaviours, and `torch.no_grad()` disables gradient computation entirely (since no weight update is needed) making the validation pass faster and more memory-efficient:

```
def evaluate(model, loader):

    # Set the model to evaluation mode
    model.eval()
```

```
# Initialize a variable to accumulate the total loss across all batches
total_loss = 0

# Disable gradient computation since we are only evaluating the model
with torch.no_grad():

    # Iterate over the batches of data provided by the loader
    for src, dec_in, dec_out in loader:

        # Move the source, decoder input, and decoder output tensors to the
        ↪ selected device
        src = src.to(device)
        dec_in = dec_in.to(device)
        dec_out = dec_out.to(device)

        # Perform a forward pass through the
        logits = model(src, dec_in)

        # Compute the loss by comparing the predicted logits with the actual
        ↪ decoder output
        loss = cross_entropy_loss(logits, dec_out)

        # Accumulate the loss for the current batch into the total loss variable
        total_loss += loss.item()

    # After processing all batches, compute the average loss by dividing the total
    ↪ loss by
    return total_loss / len(loader)
```

We train for some epochs, recording train and validation loss at each step. Since training can be time-consuming, we cache the model to disk so subsequent runs can skip training and reload the checkpoint directly:

```
import os

# Define the number of epochs for training
num_epochs = 15

# Initialize lists to store training and validation losses for each epoch
train_losses, val_losses = [], []
```

```
# Define the path to the model checkpoint file
MODEL_PATH = "./data/seq2seq_model.pt"

# Check if a model checkpoint exists at the specified path
if os.path.exists(MODEL_PATH):
    # Load the model checkpoint from the file, mapping it to the selected device
    checkpoint = torch.load(MODEL_PATH, map_location=device)
    seq2seq_model.load_state_dict(checkpoint["model_state"])
    train_losses = checkpoint["train_losses"]
    val_losses = checkpoint["val_losses"]
    num_epochs = len(train_losses)
    seq2seq_model.eval()
    print(f"Model loaded from {MODEL_PATH}")

# Otherwise, if no checkpoint exists, run the training loop and save the model to disk
else:

    # Run the training loop for the specified number of epochs
    for epoch in range(1, num_epochs + 1):

        # Perform one epoch of training and compute the training loss
        train_loss = train(seq2seq_model, eng_ita_train_loader)

        # Perform evaluation on the validation set and compute the validation loss
        val_loss = evaluate(seq2seq_model, eng_ita_val_loader)

        # Append the training and validation losses for the current epoch to their
        ↪ respective lists
        train_losses.append(train_loss)
        val_losses.append(val_loss)

        # Provide a real-time update of the training progress
        print(f"Epoch {epoch:2d} | train loss {train_loss:.4f} | val loss
        ↪ {val_loss:.4f}", end='\r', flush=True)

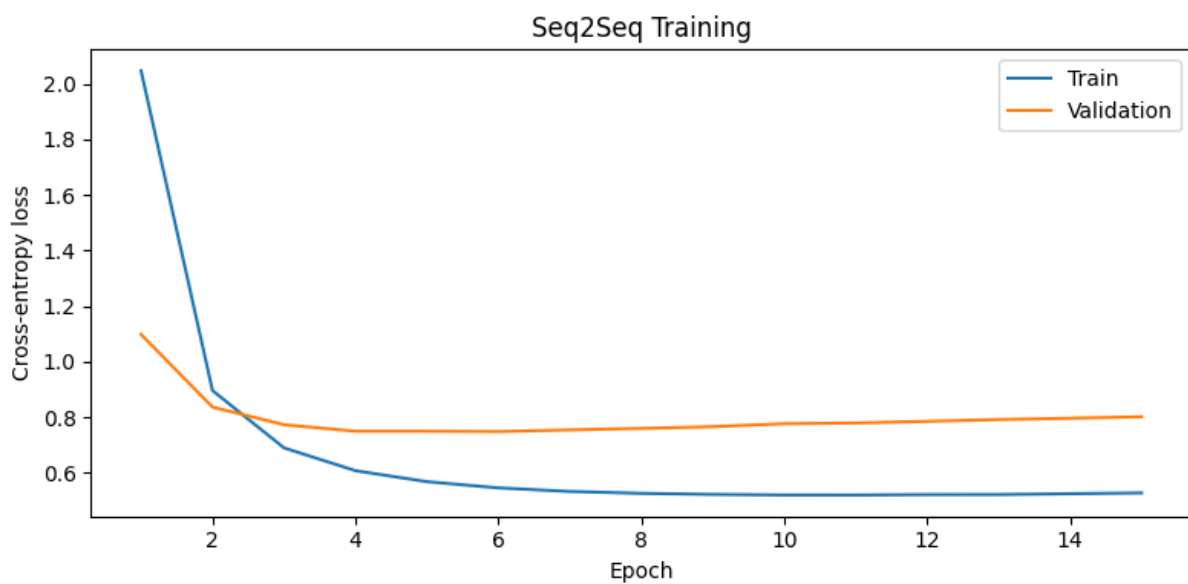
    # Save the trained model's state along with the training and validation losses
    # to a checkpoint file for future use
    torch.save({
        "model_state": seq2seq_model.state_dict(),
        "train_losses": train_losses,
        "val_losses": val_losses,
    }, MODEL_PATH)
    print(f"\nModel saved to {MODEL_PATH}")
```

Epoch 15 | train loss 0.5269 | val loss 0.8011

Model saved to ./data/seq2seq_model.pt

We plot the learning curves (training and validation loss over epochs) to visually diagnose the training dynamics: ideally both curves should decrease steadily and converge close together; a large and growing gap between them would signal **overfitting**, while both curves remaining high would suggest **underfitting**:

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(range(1, num_epochs + 1), train_losses, label='Train')
ax.plot(range(1, num_epochs + 1), val_losses, label='Validation')
ax.set_xlabel('Epoch')
ax.set_ylabel('Cross-entropy loss')
ax.set_title('Seq2Seq Training')
ax.legend()
plt.tight_layout()
plt.show()
```



Both losses drop sharply in the first two epochs, then the training loss continues to decrease slowly and steadily while the validation loss flattens. The growing gap between the two curves is a mild sign of overfitting (the model keeps improving on the training data but stops generalising beyond epoch 3 or so). Techniques such as early stopping, stronger dropout, or reducing the model capacity could help close this gap.

1.5.6 Prediction

During inference, **the target sequence is not available**, so the decoder cannot rely on the true previous token as it does during training with teacher forcing. Instead, the sequence must be generated **autoregressively**. At each time step, the decoder receives as input the token that it predicted at the previous step, rather than the ground-truth token. The generation process begins by feeding the special beginning-of-sequence token "" into the decoder at the first time step. The decoder then produces a probability distribution over the target vocabulary, and a **simple decoding strategy is to select the token with the highest probability**. This predicted token is used as the input for the next decoding step, and the procedure continues iteratively. The generation stops when the decoder produces the special end-of-sequence token "eos", indicating that the output sequence is complete.

```
def predict(model, src_sequence, eng_to_idx, ita_to_idx, idx_to_ita, max_len=20,
           device='cpu'):
    model.eval()
    with torch.no_grad():

        # encode the source sequence
        unk, pad = eng_to_idx['<unk>'], eng_to_idx['<pad>']
        src_ids = [eng_to_idx.get(t, unk) for t in src_sequence][:max_len]
        src_ids += [pad] * (max_len - len(src_ids))
        src = torch.tensor(src_ids, dtype=torch.long).unsqueeze(0).to(device)

        # encode and initialise the decoder hidden state
        _, h = model.encoder(src)

        # start decoding from <bos>
        token = torch.tensor([[ita_to_idx['<bos>']]], dtype=torch.long).to(device)
        output = []

        for _ in range(max_len):
            logits, h = model.decoder(token, h)  # (1, 1, vocab_size)
            token = logits.argmax(dim=-1)      # greedy decoding
            word = idx_to_ita[token.item()]
            if word == '<eos>':
                break
            output.append(word)

    return output
```

We evaluate the model on a few hand-picked English sentences and inspect the Italian output:

```
examples = [
    ["i", "love", "you", "."],
    ["the", "cat", "is", "on", "the", "table", "."],
    ["where", "is", "the", "train", "station", "?"],
    ["good", "morning", "!"],
    ["she", "reads", "a", "book", "every", "day", "."],
    ["we", "are", "happy", "."],
    ["what", "time", "is", "it", "?"],
    ["can", "you", "help", "me", "?"],
    ["this", "is", "a", "test", "."],
]

for src in examples:
    translation = predict(seq2seq_model, src, eng_to_idx, ita_to_idx, idx_to_ita,
↪ device=device)
    print(f"  EN: {' '.join(src)}")
    print(f"  IT: {' '.join(translation)}")
    print()
```

EN: i love you .

IT: ti amo .

EN: the cat is on the table .

IT: il gatto è sdraiato sul tavolo .

EN: where is the train station ?

IT: dov ' è la stazione ferroviaria ?

EN: good morning !

IT: buongiorno !

EN: she reads a book every day .

IT: lei legge un libro ogni giorno .

EN: we are happy .

IT: siamo felici .

EN: what time is it ?

IT: a che ora è ?

EN: can you help me ?
IT: mi potete aiutare ?

EN: this is a test .
IT: questo è un esame .

1.5.7 Performance

To get a quantitative picture, we run the model over the entire validation set and compute the BLEU score:

```
targets, predictions = [], []

for idx in eng_ita_val_set.indices:
    src = eng[idx][:-1]
    ref = ita[idx][:-1]
    pred = predict(
        seq2seq_model, src,
        eng_to_idx, ita_to_idx, idx_to_ita,
        device=device
    )
    targets.append(ref)
    predictions.append(pred)

seq2seq_bleu = bleu(predictions, targets, N=4)

print(f"BLEU: {seq2seq_bleu:.4f}")
```

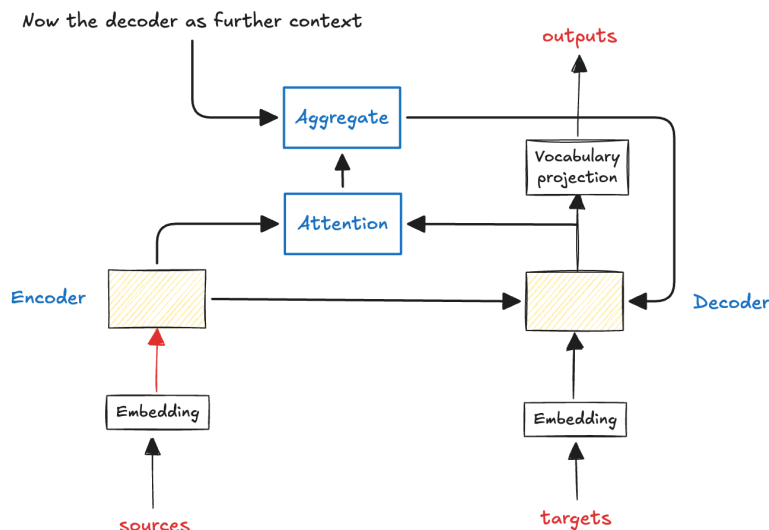
BLEU: 0.4896

That BLEU means roughly half the predicted n-grams match the reference, a solid baseline for a simple RNN trained on short sentence pairs, and a useful starting point before moving to more expressive architectures.

1.6 Overcoming the bottleneck

The encoder-decoder architecture suffers from a **fundamental limitation**. In its basic form, the encoder compresses all the information contained in the input sequence into a single fixed-

dimensional representation. This representation is then passed to the decoder, which relies on it as the sole source of information for generating the output sequence. In this sense, the intermediate state is implicitly treated as a **sufficient statistic** of the entire input:

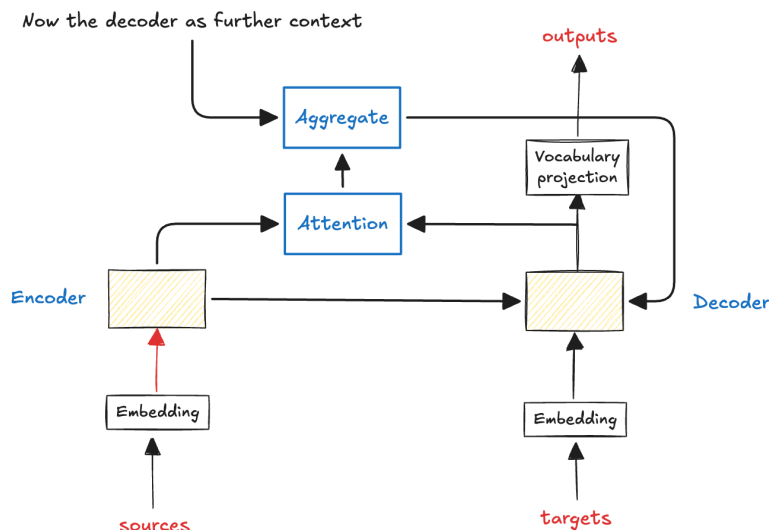


While this abstraction is **reasonable when the input is short** and the amount of information to be retained is limited, it becomes problematic as the input grows in length or complexity. It arises whenever the input is structured, variable in size, or information-rich, such as long sentences, audio signals, or high-resolution images. A fixed-size representation must compress all relevant details into a limited number of dimensions, inevitably leading to information loss. As a result, the decoder is forced to **operate on an incomplete summary of the input**, which can degrade performance.

1.6.1 Adding attention

This limitation naturally leads to the introduction of **attention mechanisms**, which can help in learning **alignment** between the input and the output. When translating a sentence, a human reader does not rely on a single compressed summary of the entire input. Instead, while generating each word, they implicitly attend to **specific parts** of the source sentence that are **most relevant** at that moment. For instance, when producing a verb, the focus typically shifts to the corresponding verb in the source, when generating a noun, attention aligns with the appropriate noun. In this sense, translation involves a **dynamic, context-dependent correspondence** between positions in the input and positions in the output. The key idea behind attention is to make this notion of alignment learnable and differentiable. Rather than forcing the decoder to depend on a single fixed representation, the model is allowed to revisit the entire input sequence at every decoding step. When predicting the next token, the decoder computes a set of

weights over the input positions, determining how much attention to pay to each part of the source sequence. These weights define a soft alignment and are used to form a context vector, which aggregates the most relevant information for the current prediction:



Concretely, instead of a fixed context variable:

$$c = h_T$$

with attention, the state becomes dynamic, at each decoding step t it is recomputed as a weighted combination of all encoder hidden states:

$$c_t = \sum_{i=1}^T \alpha_{t,i} h_i$$

The weights measure how relevant each position of the source (i) is for generating the token at each positions of the target (t). The score that produce the weights can be any of the scoring functions we have discussed, where the query is the decoder's previous hidden state (s) and keys and values are the encoder's hidden states (h):

$$e_{t,i} = w_0^\top \tanh(W_q s_{t-1} + W_k h_i)$$

For example, we can use the additive score to compute the attention weights:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^T \exp(e_{t,j})}$$

Our current implementation of the encoder already returns both the output and the state, so it can be used as it is:

```

output, state = encoder(torch.tensor([[eng_to_idx['the'],
                                       eng_to_idx['cat'],
                                       eng_to_idx['sat']]]).to(device))

# output: (batch, T, hidden_dim) ← all h_t, now we use these
# state: (num_layers, batch, hidden_dim) ← only h_T, used so f

```

The decoder needs to be modified to compute the attention weights and the context vector at each decoding step, and to use the context vector as an additional input when predicting the next token:

```

class AttentionDecoder(nn.Module):

    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers, dropout=0.0):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)

        # attention module to compute context vector c_t at each step
        self.attention = AdditiveAttention(d_q=hidden_dim, d_k=hidden_dim,
                                           h=hidden_dim)

        # input = embedding + context vector concatenated
        self.rnn = nn.GRU(embed_dim + hidden_dim, hidden_dim, num_layers,
                           batch_first=True,
                           dropout=dropout if num_layers > 1 else 0)

        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, Y, state, enc_output):

        # calculate the embedding of the input tokens
        emb = self.embedding(Y)

        # buffer to store the output of each time step
        outputs = []

        # iterate over the time steps of the target sequence
        for t in range(emb.shape[1]):

            # the query is the last output of the decoder
            query = state[-1].unsqueeze(1)

```

```
# Keys and values are the encoder outputs at all time steps
context = self.attention(query, enc_output, enc_output)

# concatenate embedding and context
rnn_input = torch.cat([emb[:, t:t+1, :], context], dim=-1)

# pass through the RNN and update the state
output, state = self.rnn(rnn_input, state)

# store the output for this time step
outputs.append(output)

# concatenate the outputs of all time steps
outputs = torch.cat(outputs, dim=1)

# pass through the final fully connected layer
logits = self.fc(outputs)

# return the logits and the final state
return logits, state
```

We now wrap encoder and decoder into a single module, mirroring the structure of Seq2Seq, but passing the full sequence of encoder hidden states to the decoder at every step, so that attention can dynamically compute a context vector rather than relying on the fixed final state:

```
class AttentionSeq2Seq(nn.Module):

    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder

    def forward(self, X, Y):
        # encode the source sequence and get the encoder outputs and final state
        enc_output, state = self.encoder(X)

        # decode the target sequence using the encoder outputs for attention
        logits, _ = self.decoder(Y, state, enc_output)

        # return the logits
        return logits
```

The following prediction function mirrors the one used for the baseline model, with two differences: the encoder output is passed to the decoder at every step to enable attention, and after each decoding step we collect the attention weights, returning them stacked as a matrix that we can use to visualise which source tokens the model focused on when generating each target token:

```
def attention_predict(model, src_sequence, eng_to_idx, ita_to_idx, idx_to_ita,
    ↪ max_len=20, device='cpu'):
    model.eval()
    with torch.no_grad():

        unk, pad = eng_to_idx['<unk>'], eng_to_idx['<pad>']
        src_ids = [eng_to_idx.get(t, unk) for t in src_sequence][:max_len]
        src_ids += [pad] * (max_len - len(src_ids))
        src = torch.tensor(src_ids, dtype=torch.long).unsqueeze(0).to(device)

        enc_output, h = model.encoder(src)

        token = torch.tensor([[ita_to_idx['<bos>']]],
    ↪ dtype=torch.long).to(device)
        output = []
        all_weights = []

        for _ in range(max_len):
            logits, h = model.decoder(token, h, enc_output)
            token = logits.argmax(dim=-1)
            word = idx_to_ita[token.item()]
            if word == '<eos>':
                break
            output.append(word)
            all_weights.append(model.decoder.attention.weights.squeeze().cpu())

    return output, torch.stack(all_weights, dim=0) # (T_tgt, T_src)
```

We instantiate the attention model with the same hyperparameters used for the baseline, so that any difference in performance can be attributed to the attention mechanism rather than to a larger or different architecture:

```
attention_encoder = Encoder(vocab_size=len(eng_to_idx), embed_dim=256,
    ↪ hidden_dim=512, num_layers=2, dropout=0.3)
attention_decoder = AttentionDecoder(vocab_size=len(ita_to_idx), embed_dim=256,
    ↪ hidden_dim=512, num_layers=2, dropout=0.3)
```

```

attention_model      = AttentionSeq2Seq(attention_encoder,
    ↪ attention_decoder).to(device)

attention_optimizer = torch.optim.Adam(attention_model.parameters(), lr=1e-3)

total_params = sum(p.numel() for p in attention_model.parameters() if p.requires_grad)
print(f"Attention model parameters: {total_params:,}")

```

Attention model parameters: 26,319,810

We train the attention model for some epochs using the same procedure, optimizer, and criterion as the baseline, caching the weights and loss curves to disk so that subsequent runs skip training and reload the checkpoint directly:

```

ATT_MODEL_PATH = "./data/attention_seq2seq_model.pt"

if os.path.exists(ATT_MODEL_PATH):
    checkpoint      = torch.load(ATT_MODEL_PATH, map_location=device)
    attention_model.load_state_dict(checkpoint["model_state"])
    attention_train_losses = checkpoint["train_losses"]
    attention_val_losses   = checkpoint["val_losses"]
    attention_num_epochs   = len(attention_train_losses)
    attention_model.eval()
    print(f"Model loaded from {ATT_MODEL_PATH}")
else:
    attention_num_epochs = 15
    attention_train_losses, attention_val_losses = [], []

    for epoch in range(1, attention_num_epochs + 1):
        train_loss = train(attention_model, eng_ita_train_loader,
    ↪ attention_optimizer, criterion, device)
        val_loss   = evaluate(attention_model, eng_ita_val_loader, criterion, device)

        attention_train_losses.append(train_loss)
        attention_val_losses.append(val_loss)

        print(f"Epoch {epoch:2d} | train loss {train_loss:.4f} | val loss
    ↪ {val_loss:.4f}", end='\r', flush=True)

    torch.save({

```

```

    "model_state": attention_model.state_dict(),
    "train_losses": attention_train_losses,
    "val_losses": attention_val_losses,
}, ATT_MODEL_PATH)

print(f"\nModel saved to {ATT_MODEL_PATH}")

```

Epoch 15 | train loss 0.6195 | val loss 0.7775

Model saved to ./data/attention_seq2seq_model.pt

We plot the training and validation loss curves for both models side by side, sharing the y-axis to make the comparison direct:

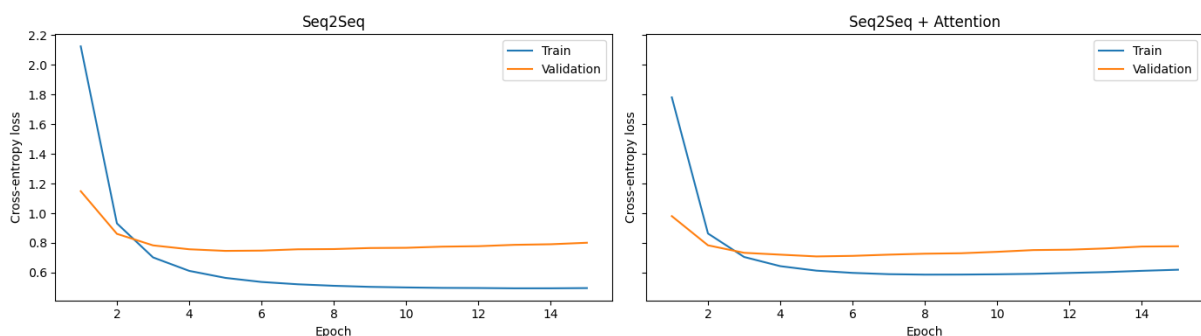
```

fig, axes = plt.subplots(1, 2, figsize=(14, 4), sharey=True)

for ax, tr, va, ne, title in [
    (axes[0], train_losses, val_losses, num_epochs, 'Seq2Seq'),
    (axes[1], attention_train_losses, attention_val_losses, attention_num_epochs,
      'Seq2Seq + Attention'),
]:
    ax.plot(range(1, ne + 1), tr, label='Train')
    ax.plot(range(1, ne + 1), va, label='Validation')
    ax.set_title(title)
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Cross-entropy loss')
    ax.legend()

plt.tight_layout()
plt.show()

```



Both models converge quickly within the first few epochs and then plateau. The baseline shows a wider and growing gap between train and validation loss, a sign of mild overfitting. The

attention model keeps the two curves closer together for longer, suggesting that **the additional context acts as a regulariser**.

We prepare a prediction function which mirrors the one used for the baseline model, with two differences: the encoder output is passed to the decoder at every step to enable attention, and after each decoding step we collect the attention weights, returning them stacked as a matrix that we can use to visualise which source tokens the model focused on when generating each target token:

```
def attention_predict(model, src_sequence, eng_to_idx, ita_to_idx, idx_to_ita,
    ↪ max_len=20, device='cpu'):
    model.eval()
    with torch.no_grad():
        unk, pad = eng_to_idx['<unk>'], eng_to_idx['<pad>']
        src_ids = [eng_to_idx.get(t, unk) for t in src_sequence][:max_len]
        src_ids += [pad] * (max_len - len(src_ids))
        src = torch.tensor(src_ids, dtype=torch.long).unsqueeze(0).to(device)

        enc_output, h = model.encoder(src)

        token, output, all_weights = torch.tensor([[ita_to_idx['<bos>']]],
    ↪ dtype=torch.long).to(device), [], []

        for _ in range(max_len):
            logits, h = model.decoder(token, h, enc_output)
            token = logits.argmax(dim=-1)
            word = idx_to_ita[token.item()]
            if word == '<eos>':
                break
            output.append(word)
            all_weights.append(model.decoder.attention.weights.squeeze().cpu())

    return output, torch.stack(all_weights, dim=0) # (T_tgt, T_src)
```

We evaluate the model on the validation set using the same BLEU metric computed earlier, so we can directly compare translation quality before and after introducing attention:

```
targets, attention_preds = [], []

for idx in eng_ita_val_set.indices:
    src = eng[idx][:-1]
    ref = ita[idx][:-1]
```

```

    pred, _ = attention_predict(
        attention_model, src,
        eng_to_idx, ita_to_idx, idx_to_ita,
        device=device
    )
    targets.append(ref)
    attention_preds.append(pred)

attention_bleu = bleu(attention_preds, targets, N=4)

print(f"Seq2Seq BLEU: {seq2seq_bleu:.4f}")
print(f"Attention BLEU: {attention_bleu:.4f}")

```

Seq2Seq BLEU: 0.4998

Attention BLEU: 0.5053

The attention model scores marginally higher, but the difference is negligible. This is not surprising: the dataset consists mostly of short sentences where a fixed context vector is already sufficient, so the dynamic has little room to show its advantage. Attention becomes significantly more impactful on longer, more complex sequences. Beyond aggregate metrics, we can inspect a few hand-picked sentences to get a qualitative feel for where attention helps and where both models still struggle:

```

examples = [
    ["i", "love", "you", "."],
    ["the", "cat", "is", "on", "the", "table", "."],
    ["where", "is", "the", "train", "station", "?"],
    ["she", "reads", "a", "book", "every", "day", "."],
]

for src in examples:
    base = predict(seq2seq_model, src, eng_to_idx, ita_to_idx, idx_to_ita,
    ↪ device=device)
    attention, _ = attention_predict(attention_model, src, eng_to_idx, ita_to_idx,
    ↪ idx_to_ita, device=device)
    print(f"  EN:                {' '.join(src)}")
    print(f"  Seq2Seq:                {' '.join(base)}")
    print(f"  Seq2Seq + Attention: {' '.join(attention)}")
    print()

```

EN: i love you .

```
Seq2Seq:          ti amo .
Seq2Seq + Attention: io ti amo .

EN:               the cat is on the table .
Seq2Seq:          il gatto è sul tavolo .
Seq2Seq + Attention: il gatto è sul tavolo .

EN:               where is the train station ?
Seq2Seq:          dov ' è la stazione ferroviaria ?
Seq2Seq + Attention: dov ' è la stazione dei treni ?

EN:               she reads a book every day .
Seq2Seq:          legge ogni libro ogni giorno .
Seq2Seq + Attention: legge un libro ogni giorno .
```

Both models perform similarly on simple, short sentences, where the fixed context vector already captures enough information. The difference emerges on slightly harder cases: for “she reads a book every day” the baseline produces the **hallucinated** “ogni libro” (every book) instead of “un libro” (a book), while attention corrects it by being able to focus on “a” independently when generating that token. Overall the qualitative improvement is modest, consistent with what the BLEU scores showed: attention helps at the margin but the task remains well within reach of the baseline for short, frequent sentences. Finally, we create a function to **visualise the weight matrix**, letting us inspect which source tokens the decoder attended to when generating each target word:

```
def show_attention(matrix, src_tokens, tgt_tokens):
    fig, ax = plt.subplots(figsize=(len(src_tokens) * 0.9, len(tgt_tokens) * 0.9 + 1))

    pcm = ax.imshow(matrix.detach().numpy(), cmap='Blues')

    ax.set_xticks(range(len(src_tokens)))
    ax.set_xticklabels(src_tokens, rotation=45, ha='right')
    ax.set_yticks(range(len(tgt_tokens)))
    ax.set_yticklabels(tgt_tokens)

    ax.set_xlabel('Source (keys)')
    ax.set_ylabel('Target (queries)')
    ax.set_title('Attention weights')
```

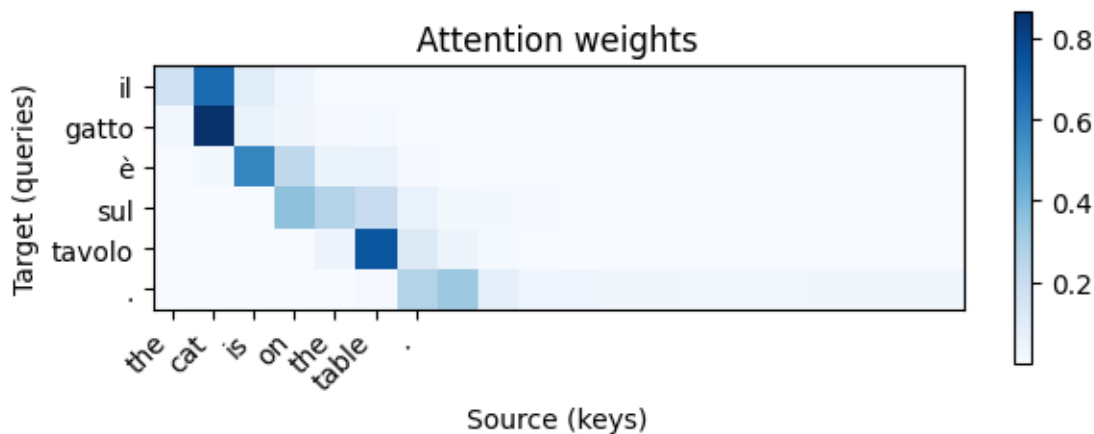
```
fig.colorbar(pcm, ax=ax, shrink=0.3)
plt.tight_layout()
plt.show()
```

We run the prediction on a simple sentence and pass the returned weight matrix directly to function to produce the heatmap:

```
src = ["the", "cat", "is", "on", "the", "table", "."]

translation, weights = attention_predict(
    attention_model, src, eng_to_idx, ita_to_idx, idx_to_ita, device=device
)

show_attention(weights, src_tokens=src, tgt_tokens=translation)
```



The heatmap shows a clear near-diagonal pattern: each Italian token attends most strongly to its direct English counterpart ("gatto" and "cat", "è" and "is", "tavolo" and "table") which is expected for a language pair as close as English and Italian. The most notable deviation is "il", which spreads attention across both "the" and "cat", reflecting the fact that the Italian definite article must agree with the noun it precedes and so the model looks ahead to determine gender. And also "sul" xxxx

1.6.2 Self-attention

1.7 The transformer architecture